

hkw2/alush2.vhd

```

1  -----
2  --
3  --  SH-2 ALU
4  --
5  --  This is an implementation of an ALU for the SH-2 it uses the
6  --  generic ALU. The ALU implements an F block, adder and shifter and MUX
7  --  between all results. Multiply/Divide instructions and MAC instructions are
8  --  not implemented. Shift instructions by 2, 6, and 8 are implemented. Swap
9  --  byte instructions, sign and zero extension instructions, and extract
10 --  instructions are also implemented.
11 --
12 --  Packages included are:
13 --      TConstants - constants for how the Tbit should be set based on the instruction
14 --
15 --  Entities included are:
16 --      ShiftN - Shift and rotate by N bits (generic shiftamt)
17 --      SwapB - swap bytes 0 and 1 with bytes 2 and 3
18 --      Extend - sign and zero extension by byte or word
19 --      ALUSH2 - the ALU for SH2
20 --
21 --  Revision History:
22 --      7 Apr 26 Simone Shevchuk Initial revision.
23 --      9 Apr 26 Simone Shevchuk Added Tbit Zero and one set, update XNOR
24 --      15 Apr 26 Simone Shevchuk Changed from 3 shifters to ShiftN, add Swap, Sign extension, and extract
25 --      22 Apr 26 Simone Shevchuk Update Tbit logic, add zero pre-ALU for TAS instruction
26 --      12 May 26 Simone Shevchuk Update Tbit to not feed itself
27  -----
28
29  library ieee;
30  use ieee.std_logic_1164.all;
31
32  -- Tbit command constants, used to select which value to set the Tbit to based on the instruction
33  package TConstants is
34      constant TCmd_HOLD : std_logic_vector(3 downto 0) := "0000";
35      constant TCmd_C : std_logic_vector(3 downto 0) := "0001"; -- carry
36      constant TCmd_V : std_logic_vector(3 downto 0) := "0010"; -- overflow
37      constant TCmd_Z : std_logic_vector(3 downto 0) := "0011"; -- zero
38      constant TCmd_S : std_logic_vector(3 downto 0) := "0100"; -- sign
39      constant TCmd_CNZ : std_logic_vector(3 downto 0) := "0101"; -- carry and not zero
40      constant TCmd_SXNORV : std_logic_vector(3 downto 0) := "0110"; -- not (sign xor overflow)
41      constant TCmd_SXNORVNZ : std_logic_vector(3 downto 0) := "0111"; -- not (sign xor overflow) and not zero
42      constant TCmd_NSNZ : std_logic_vector(3 downto 0) := "1000"; -- not (negative or zero)
43      constant TCmd_NS : std_logic_vector(3 downto 0) := "1001"; -- not sign
44      constant TCmd_LSB : std_logic_vector(3 downto 0) := "1010"; -- least significant bit of the result
45      constant TCmd_CMPSTR : std_logic_vector(3 downto 0) := "1011"; -- result of compare str instruction, set to 1
46      if any byte of the result is 0, otherwise 0
47      constant TCmd_NC : std_logic_vector(3 downto 0) := "1100"; -- not carry
48      constant TCmd_ZERO : std_logic_vector(3 downto 0) := "1101"; -- set to 0, override other settings
49      constant TCmd_ONE : std_logic_vector(3 downto 0) := "1110"; -- set to 1, override other settings
50      constant TCmd_ZPALU : std_logic_vector(3 downto 0) := "1111"; -- is data data bus value zero, before ALU,
51      used for TAS
52
53  end package;
54
55  --
56  --  ShiftN
57  --
58  --  Shifter for doing shift/rotate operations by N in {2, 8, 16} bits in the ALU.
59  --  The determination of shift amount is given by an input ShiftSel, and the
60  --  type of shift (only relevant as left or right) is given by SCmd.
61  --
62  --  Inputs:
63  --      S0p - operand
64  --      ShiftSel - shift to apply to offset, 0 for shift of 2, 1 for shift of 8, 2 for shift of 16

```

```

64 -- SCmd      - operation to perform (3 bits, SCmd_LEFT / SCmd_RIGHT)
65 --
66 -- Outputs:
67 -- SResult - shift result
68 --
69
70 library ieee;
71 use ieee.std_logic_1164.all;
72 use ieee.numeric_std.std_match;
73 use work.ALUConstants.all;
74
75 entity ShiftN is
76     port (
77         S0p      : in  std_logic_vector(31 downto 0);
78         SCmd      : in  std_logic_vector(2 downto 0); -- same encoding as the SCmd for shifts in the generic ALU, but
only LEFT and RIGHT are relevant
79         ShiftSel  : in  integer range 2 downto 0; -- 0 to 2, 1 to 8, 2 to 16
80         SResult   : out std_logic_vector(31 downto 0)
81     );
82 end ShiftN;
83
84 architecture dataflow of ShiftN is
85 begin
86     -- use concatenation and slicing to implement the shifts/rotates by the appropriate amount based on the command
and select signal
87     SResult <=
88         b"00" & S0p(31 downto 2) when ShiftSel=0 and std_match(SCmd,SCmd_RIGHT) else
89         S0p(29 downto 0) & b"00" when ShiftSel=0 and std_match(SCmd,SCmd_LEFT) else
90         b"00000000" & S0p(31 downto 8) when ShiftSel=1 and std_match(SCmd,SCmd_RIGHT) else
91         S0p(23 downto 0) & b"00000000" when ShiftSel=1 and std_match(SCmd,SCmd_LEFT) else
92         x"0000" & S0p(31 downto 16) when ShiftSel=2 and std_match(SCmd,SCmd_RIGHT) else
93         S0p(15 downto 0) & x"0000" when ShiftSel=2 and std_match(SCmd,SCmd_LEFT) else
94         (others => 'X');
95 end dataflow;
96
97
98 --
99 -- SwapB
100 --
101 -- This is the swapper for doing byte swap operations in the ALU. Unlike the swap operation
102 -- in the generic ALU, this only swaps bytes 0 and 1 with bytes 2 and 3, it does
103 -- not swap other bytes/whole words.
104 --
105 --
106 -- Inputs:
107 -- S0p      - operand
108 --
109 -- Outputs:
110 -- SResult - swap result
111 --
112
113 library ieee;
114 use ieee.std_logic_1164.all;
115
116 entity SwapB is
117     port(
118         S0p      : in  std_logic_vector(31 downto 0); -- operand
119         SResult   : out std_logic_vector(31 downto 0) -- swap result
120     );
121
122
123 end SwapB;
124
125
126 architecture dataflow of SwapB is
127 begin
128

```

```

129     -- swap bytes 0 and 1 of the low word; upper word is preserved unchanged
130     SResult(7 downto 0)  <= S0p(15 downto 8);
131     SResult(15 downto 8) <= S0p(7 downto 0);
132     SResult(31 downto 16) <= S0p(31 downto 16);
133
134 end   dataflow;
135
136 --
137 -- Extend
138 --
139 -- This is the extender for doing sign or zero extension operations in the ALU.
140 -- The extension is defined with an input for the type of extension to perform.
141 -- This can be sign extending by byte or by word, or zero extending by byte or by word.
142 --
143 --
144 -- Inputs:
145 --     S0p      - operand
146 --     ExtendSel - extension command (2 bits)
147 --
148 -- Outputs:
149 --     SResult - swap result
150 --
151
152 library ieee;
153 use ieee.std_logic_1164.all;
154
155 entity Extend is
156     port(
157         S0p      : in  std_logic_vector(31 downto 0);    -- operand
158         ExtendSel : in  std_logic_vector(1 downto 0);    -- extension command
159         -- (00 for sign extend byte, 01 for sign extend word, 10 for zero extend byte, 11 for zero extend word)
160         EResult   : out std_logic_vector(31 downto 0)    -- extension result
161     );
162
163 end   Extend;
164
165
166 architecture dataflow of Extend is
167 begin
168
169     -- for sign extension, if the high bit of the byte/word is 1, set the upper bits to 1, otherwise set to 0
170     EResult <= (31 downto 8 => S0p(7))    &  S0p(7 downto 0)    when ExtendSel = "00" else
171                (31 downto 16 => S0p(15)) &  S0p(15 downto 0) when ExtendSel = "01" else
172                (31 downto 8 => '0')      &  S0p(7 downto 0)    when ExtendSel = "10" else
173                (31 downto 16 => '0')      &  S0p(15 downto 0) when ExtendSel = "11" else
174                (others => '0'); -- for zero extension just set upper bits to 0
175
176 end   dataflow;
177
178
179 --
180 -- ALUSH2
181 --
182 -- This is an ALU for SH2. The SH-2 ALU uses the generic ALU implementation
183 -- This generates appropriate signals from the control signals and creates a
184 -- shifter, swapper and etender to calculate the result of instructions that
185 -- are not covered by the generic ALU (SWAP.B, SHLLN, EXTU). The final result
186 -- muxes between the generic ALU result, shifter, swapper, extender and extract
187 -- based on the command. The Tbit is set based on the command to either be the
188 -- carry out from the operation, the overflow, the zero result, the sign result,
189 -- or some combination of these.
190 --
191 --
192 -- Inputs:
193 --     alu_opa - first operand
194 --     alu_opb - second operand
195 --     c_in    - carry in (from tbit)

```

```

196 -- f_cmd      - f-block operation to perform (4 bits)
197 -- c_in_cmd   - adder carry in operation for carry in (2 bits)
198 -- s_cmd      - shift operation to perform (3 bits)
199 -- alu_cmd    - alu operation to perform - selects result (2 bits)
200 -- shift_sel  - shift amount selection for shift by N unit (0 for shift by 2, 1 for shift by 8, 2 for shift by 16)
201 -- extend_sel- extension operation selection for extender unit (2 bits)
202 -- res_sel    - selects between alu result, shift by N result, swap result, extend result, and extract result
203 -- t_sel      - select for how the tbit should be set
204 --
205 -- Outputs:
206 -- res        - ALUSH2 result
207 -- t_bit      - Tbit ouputut as one of the constants above
208
209 library ieee;
210 use ieee.std_logic_1164.all;
211 use work.ALUConstants.all;
212 use work.TConstants.all;
213
214 entity ALUSH2 is
215
216     port(
217         alu_opa : in     std_logic_vector(31 downto 0); -- first operand
218         alu_opb : in     std_logic_vector(31 downto 0); -- second operand
219         c_in    : in     std_logic;                    -- carry in
220         f_cmd   : in     std_logic_vector(3 downto 0); -- F-Block operation
221         c_in_cmd : in     std_logic_vector(1 downto 0); -- carry in operation
222         s_cmd   : in     std_logic_vector(2 downto 0); -- shift operation
223         alu_cmd : in     std_logic_vector(1 downto 0); -- Generic ALU result select
224         shift_sel : in    integer range 2 downto 0;    -- shift amount selection
225         extend_sel : in   std_logic_vector(1 downto 0); -- extension operation selection
226         res_sel : in     integer range 4 downto 0;     -- select between shiftN, swap, extend, extract and ALU
227
228         result
229         t_sel   : in     std_logic_vector(3 downto 0); -- select for the Tbit
230         res     : buffer std_logic_vector(31 downto 0); -- ALUSH2 result
231         t_bit   : out    std_logic                    -- Tbit output, see constants above
232     );
233
234 end ALUSH2;
235
236 architecture structural of ALUSH2 is
237
238     signal GenALURes      : std_logic_vector(31 downto 0); -- generic ALU result
239     signal ShiftNRes      : std_logic_vector(31 downto 0); -- shift-by-N result
240     signal swap           : std_logic_vector(31 downto 0); -- swap result
241     signal extend         : std_logic_vector(31 downto 0); -- extend result
242     signal extract        : std_logic_vector(31 downto 0); -- extract result
243
244     signal c_out          : std_logic;                    -- carry out from the operation, used for Tbit
245     signal overflow_res   : std_logic;                    -- overflow from the operation, used for Tbit
246     signal zero_res       : std_logic;                    -- result of comparison is value 0, used for Tbit
247     signal sign_res       : std_logic;                    -- result of comparison is negative, used for Tbit
248     signal cmp_str_res    : std_logic;                    -- result of compare str instruction, used for Tbit
249     signal zero_prealu    : std_logic;                    -- is the data data bus value zero before the ALU,
250
251     TAS instruction
252
253 begin
254     -- instantiate the generic ALU
255     ALUGeneric: entity work.ALU
256     generic map (
257         wordsize => 32 -- 32-bit wide
258     )
259
260     port map (
261         ALUOpA => alu_opa,
262         ALUOpB => alu_opb,
263         Cin    => c_in,

```

```

261         FCmd      => f_cmd,
262         CinCmd    => c_in_cmd,
263         SCmd      => s_cmd,
264         ALUCmd    => alu_cmd,
265         Result    => GenALURes,
266         Cout      => c_out,
267         HalfCout  => open,
268         Overflow  => overflow_res,
269         Zero      => zero_res,
270         Sign      => sign_res
271     );
272
273     -- instantiate the shift-by-N units using ShiftN
274     ShiftNUnit: entity work.ShiftN
275     port map (
276         S0p => alu_opa,
277         ShiftSel => shift_sel,
278         SCmd => s_cmd,
279         SResult => ShiftNRes
280     );
281
282     -- instantiate the swap unit
283     SwapUnit: entity work.SwapB
284     port map (
285         S0p => alu_opa,
286         SResult => swap
287     );
288
289     -- instantiate the extend unit
290     ExtendUnit: entity work.Extend
291     port map (
292         S0p => alu_opa,
293         ExtendSel => extend_sel,
294         EResult => extend
295     );
296
297     -- for extract, the upper half is bits 15-0 of operand B and the lower half is bits 15-0 of operand A
298     extract(15 downto 0) <= alu_opa(31 downto 16);
299     extract(31 downto 16) <= alu_opb(15 downto 0);
300
301     -- mux between the different results based on the command to produce the final result
302     res <= ShiftNRes when (res_sel = 1) else
303         swap when (res_sel = 2) else
304         extend when (res_sel = 3) else
305         extract when (res_sel = 4) else
306         GenALURes when (res_sel = 0) else
307         (others => 'X'); -- anything else is illegal
308
309     -- for CMPSTR set to 1 if any byte of the result is 0, otherwise 0
310     cmp_str_res <= '1' when (res(31 downto 24) = x"00") or
311         (res(23 downto 16) = x"00") or
312         (res(15 downto 8) = x"00") or
313         (res(7 downto 0) = x"00")
314         else '0';
315
316     zero_prealu <= '1' when alu_opa = (alu_opa'range => '0') else '0'; -- check if data data bus value is zero
    before ALU
317
318     -- set the Tbit based on the command, using the constants defined above
319     t_bit <= c_in when (t_sel = TCmd_HOLD) else
320         c_out when (t_sel = TCmd_C) else
321         overflow_res when (t_sel = TCmd_V) else
322         zero_res when (t_sel = TCmd_Z) else
323         sign_res when (t_sel = TCmd_S) else
324         (c_out and not(zero_res)) when (t_sel = TCmd_CNZ) else
325         not (sign_res xor overflow_res) when (t_sel = TCmd_SXNORV) else
326         (not (sign_res xor overflow_res) and not(zero_res)) when (t_sel = TCmd_SXNORVNZ) else

```

```
327      (not sign_res) and (not zero_res)
328      not sign_res
329      res(0)
330      cmp_str_res
331      not c_out
332      '0'
333      '1'
334      zero_prealu
335      'X'; -- anything else is illegal
336 end structural;
```

```
when (t_sel = TCmd_NSNZ)      else
when (t_sel = TCmd_NS)       else
when (t_sel = TCmd_LSB)      else
when (t_sel = TCmd_CMPSTR)   else
when (t_sel = TCmd_NC)       else
when (t_sel = TCmd_ZERO)     else
when (t_sel = TCmd_ONE)      else
when (t_sel = TCmd_ZPALU)    else
```